

ACADEMIC INTEGRITY. You are reminded that you should understand and comply with basic standards of academic integrity –in particular, in matters related to misrepresentation by deception or by other fraudulent means of submitted work. Lack of academic integrity can result in serious consequences –e.g., the grade of zero on the assignment. The following illustrates only three forms of academic dishonesty: (1) plagiarism, e.g., the submission of work that is not one’s own or for which other credit has been obtained; (2) improper collaboration; (3) copying or using unauthorized material.

SUBMISSION. You must submit your solution using Python files with *.py*. Your solution must also include any provided files, even if you didn’t modified them. As for any additional files you want to submit, the only accepted formats are: plain text file (*.txt*); markdown files (*.md*); and PDF files (*.pdf*). You can upload your submission as a compressed file (accepted formats are *zip* and *tar.gz*).

Context

This Assignment is meant to put into practice the concepts we have seen so far. We will be using Python.

The provided code allows to represent boolean expressions, to construct boolean expressions, and to evaluate them. The types of boolean expressions that we will use are as follows:

- Constant: C either *True* or *False*.
- Variable: $Var\ v$ a name, that requires an interpretation to be evaluated.
- Negation: $Not\ E$ the negation of a boolean expression.
- Conjunction: $E_1\ AND\ E_2$ a conjunction of two boolean expressions.
- Disjunction: $E_1\ OR\ E_2$ a disjunction of two boolean expressions.
- Exclusive disjunction: $E_1\ XOR\ E_2$ an exclusive disjunction of two boolean expressions.
- Implication: $E_1\ IMPLIES\ E_2$ an implication of a boolean expression by another boolean expression.
- Bi-implication: $E_1\ IFF\ E_2$ a bi-implication of two boolean expressions.

The semantics of these expressions are given in the context of an interpretation I , a mapping (represented here as a list of tuples) of variable names on constant values, i.e.: *True* and *False*.

- $eval(C, I)$ evaluates to C .
- $eval(Var\ v, I)$ evaluates to $I[v]$.
- $eval(Not\ E, I)$ evaluates to $\neg eval(E, I)$.
- $eval(E_1\ AND\ E_2, I)$ evaluates to $eval(E_1, I)$ and $eval(E_2, I)$.
- $eval(E_1\ OR\ E_2, I)$ evaluates to $eval(E_1, I)$ or $eval(E_2, I)$.
- $eval(E_1\ XOR\ E_2, I)$ evaluates to $eval(E_1, I)$ xor $eval(E_2, I)$.
- $eval(E_1\ IMPLIES\ E_2, I)$ evaluates to $\neg eval(E_1, I)$ or $eval(E_2, I)$.
- $eval(E_1\ IFF\ E_2, I)$ evaluates to $eval(E_1, I) = eval(E_2, I)$.

Exercises

1. Add the expressions *IMPLIES*, *IFF*, and *XOR* to the *BinaryExpression* class, making all the necessary modification to *ExpressionBuilder*, *Solver*, and *ExpressionBuilder* classes. For *ExpressionBuilder* you must add the following functions:
 - *implicate* which takes an *Expression* (*Q*) and modifies the expression (*E*) associated with the current instance of *ExpressionBuilder* such that the associated expression becomes *E IMPLIES Q*.
 - *iff* which takes an *Expression* (*Q*) and modifies the expression (*E*) associated with the current instance of *ExpressionBuilder* such that the associated expression becomes *E IFF Q*.
 - *exclusive_disjunction* which takes an *Expression* (*Q*) and modifies the expression (*E*) associated with the current instance of *ExpressionBuilder* such that the associated expression becomes *E XOR Q*.
2. Implement all functions in class *Solver*.
3. Modify *main.py* to build all the commented expressions, for each expression you must print the following:
 - 1 The expression itself (using function *str*).
 - 2 The variables of the expression (on a single line).
 - 3 All possible interpretations (one on each line).
 - 4 If the expression is satisfiable.
 - 5 If the expression is a tautology (or not).
 - 6 If the expression is a contradiction (or not).
 - 7 All interpretations that satisfies the expression (one on each line).
 - 8 All interpretations that do not satisfy the expression (one on each line).